Maths with Python — Python reference

1. Getting started

These instructions will assume that you are comfortable with using Python in Jupyter notebooks in VS Code, and that you have installed the packages `sympy`, `numpy`, `scipy` and `plotly` using `pip` or `conda`. You will also need to import these packages and set some options. The most convenient way to do this is as follows. Distributed together with this file is a directory called `header` containing a single file `__init__.py`. You should copy this directory to be a subdirectory of the directory where you keep your notebooks for this course, then you should include the line

```
from header import *
```

as the top line of each of your notebooks.

2. Generalities

(2.1) If you know the name of a function that you want to use (for example, `sp.solve`) you can enter `help(sp.solve)` to get some documentation. You can also get a shorter version of that documentation by just typing `sp.solve` in a code cell and hovering over it with your mouse.

(2.2) See Section 15 for advice on some common problems.

(2.3) If you have several commands in the same code cell then Python will only print the result of the last one by default. If you want to see other things then you should add explicit `print()` or `display()` commands.

(2.4) If you end a command with a semicolon then Python will not print the result even if it is the last command in the cell. You can also achieve the same effect by entering `None` as the last command in the cell.

(2.5) You can use the symbol `_` to refer to the last thing that Python calculated and displayed, and `__` to refer to the one before that, and so on. However, this mechanism cannot be used to refer to things that were calculated but not displayed.

(2.6) If you move around a worksheet inserting things in different places, you should remember that `_` refers to the **most recent** result, which may not be the one that you see directly above the line where you are typing.

(2.7) It is important to understand the difference between `=` and `==` and `sp.Eq()`.
- If we enter $x = 3$, this defines x to be equal to 3, so Python will replace x by 3 whenever it sees an x.
- If we instead enter $x == 3$, we are checking whether x is currently straightforwardly equal to 3. If x is an abstract symbol which has not been assigned a numerical value then $x == 3$ will just be false, even if x could be equal to 3 in some contexts.
- We can instead enter `sp.Eq(x=3)` to represent $x = 3$ as an abstract equation which might be true or false in different circumstances. This is the right thing to do if we are setting up some equations that we later want to solve.

(2.8) If you want to define a function (say $f(x) = x^2 + 3$) it does *not* work to enter `f(x)=x^2+3;`. See Section 10.

(2.9) Near the top of your VS Code window you should see a menu item marked `Restart`. If you click this then Python will restart and all values that you might have assigned to variables will be removed. You might want to do this at the beginning of each new exercise, otherwise stray values left over from previous exercises can cause trouble and confusion. Note, however, that most notebooks will have a cell at the top with some `import` statements. Restarting removes all imports, so you will need to go back to your import cell and execute it again before continuing.

(2.10) If you just want to clear a single variable (say x) you can enter `del x` instead. To clear several variables, you can enter something like `del A,x,U`.

3. Numerical approximation

(3.1) To get an explicit numerical approximation for an expression, use the function `sp.N()` function. For example, `sp.N(sp.pi)` gives 3.14159265358979, and `sp.N(x/3)` gives $0.3333333333x$.

(3.2) The function `sp.N()` generally gives an answer to 15 significant figures. If you want to calculate $\pi^2/6$ to 50 digits (for example), you can enter `sp.N(sp.pi**2/6, 50)`.

(3.3) It is possible to lose accuracy for non-obvious reasons. For example, if you enter `sp.N(1/7, 100)` then Python will work from the inside out, first calculating $1/7$ to low accuracy and only then calling `sp.N()` with a request for high accuracy that can no longer be achieved. One way to fix this is to enter `sp.N(1,100)/7`, which requests high accuracy before attempting to divide by 7.

4. Notation for variables

(4.1) By default names like x refer to variables as used in programming languages like Python. They are expected to have a value (which might be a number or a list or something more complicated). If you try to use a variable (for example, by trying to expand $(1+x)^4$) without assigning a value then you will get an error. If you instead want to treat x as an abstract symbol then you need to enter `x = sp.symbols('x')`. You can do this for several variables at the same time using syntax like `x,y,a,b = sp.symbols('x y a b')`.

(4.2) If you want variables with indices like m_0, m_1, m_2 then you should enter `m = sp.IndexedBase('m')`, and then enter `m[2]` for m_2, for example.

(4.3) Most variables will have single-letter names, such as x or A.

(4.4) Note that case is significant: Python regards A as different from a, and x as different from X.

(4.5) It is perfectly permissible to use long names for variables, such as `eqns` or `sols` or `SpeedOfLight`.

(4.6) However, you may not use words that already have a special meaning in Python. In particular, you cannot call a variable `lambda`.

(4.7) If you enter `mu = sp.symbols('mu')` then in some (but not all) contexts, `mu` will be displayed as μ.

5. Algebraic operations

(5.1) Use a `*` for multiplication; for example, $2xy$ should be entered as `2*x*y`, and $(2x+1)\sin(3x)$ should be entered as `(2*x+1)*sp.sin(3*x)`.

(5.2) Note also that Python will treat `xy` as a two-letter name for a single variable; if you want x times y, you must enter `x*y`.

(5.3) Just as in standard mathematical notation, bracketing is important. To multiply $x+2$ by $x-2$, you must enter `(x+2)*(x-2)`, not `x+2*x-2`.

(5.4) Enter powers using `**`, for example `x ** 6` for x^6. The notation `x^6` does have a meaning (the exclusive or operator applied to x and 6) but it is almost never what you want; this can unfortunately lead to some confusing error messages.

(5.5) Just as in standard mathematical notation, bracketing is important. To raise $x+2$ to the power $n+5$, you must enter `(x+2)**(n+5)`, not `x+2**n+5`. To get the cube root of x, enter `x**(1/3)`, not `x**1/3`.

(5.6) The square root of x can be entered as `sp.sqrt(x)`. Similarly, $\sqrt{b^2-4ac}$ is `sp.sqrt(b**2-4*a*c)`. For more general roots you might be tempted to enter something like `(x+y)**(1/3)` for the cube root of $x+y$. However, it is actually necessary to enter `(x+y)**sp.Rational(1,3)` instead, to ensure that we use the exact fraction $1/3$ instead of converting it to an approximate decimal, which is what Python would do by default.

6. Algebraic manipulation

Suppose we have a complicated expression called A, which we want to simplify or otherwise manipulate.

(6.1) To substitute for some or all of the variables in A, use the `subs` command. For example, if A is $(a+b)^n$, you can change the b to 1 and the n to p/q by entering `A.subs({b=1,n=p/q})`, which gives $(a+1)^{p/q}$.

(6.2) Often the equations to be used in `subs(...)` will come from the `solve(...)` command. For example, to find the value of x^2+y^2 at the point where $x+y=6$ and $x-y=2$, enter `sol=sp.solve([sp.Eq(x+y,6),sp.Eq(x-y,2)])` and then `(x**y + y**2).subs(sol)`.

(6.3) Enter `sp.expand(A)` to expand everything out. For example, enter `sp.expand((x+y)**3)` to expand $(x+y)^3$, giving $x^3+3x^2y+3xy^2+y^3$.

(6.4) To simplify A, try entering `sp.simplify(A)`. If this does not do what you want, you can try `sp.factor(A)`. If A involves trigonometric functions, try `sp.trigsimp(A)` or `sp.expand_trig(A)`.

(6.5) Note that rules like $(a^4)^{1/4} = a$ are **not valid** when a is negative. The function `sp.simplify()` will not simplify expressions like this by default. However, if you declare x to be positive using `x = sp.symbols('x', positive=True)` (instead of just `x = sp.symbols('x')`) then sympy will apply rules that are only valid for positive x. For example, `(x ** 4) ** sp.Rational(1,4)` will simplify to x. (However, `(x ** 4) ** (1/4)` will only simplify to $x^{1.0}$, which will not simplify to x, because the exponent is only approximately equal to one and not exactly equal.) There are also more complex ways to specify that x is an integer or lies in a certain range or has other useful properties that could be used when simplifying; see the sympy documentation about assumptions.

(6.6) If you suspect that A is equal to some other expression (say B) and want to check this, the best way is to enter `sp.simplify(A-B)` and see if you get zero.

(6.7) To collect together all the terms in A involving the same power of x, enter `A.collect(x)`. For example, this converts $1 + ax + bx + cx^2 + dx^2$ to $1 + (a + b)x + (c + d)x^2$.

(6.8) If you just want the coefficient of x^2 in A then you can instead enter `A.coeff(x,2)` or `A.coeff(x**2)`. To get the constant term, you must enter `A.coeff(x,0)` — the syntax `A.coeff(x**0)` will not work.

7. Constants

(7.1) If you enter a fraction like `2/9` then it will immediately be converted to an approximate decimal, which is not usually what you want. If you want to treat it as an exact fraction you should enter `sp.Rational(2,9)` instead.

(7.2) The constant $\pi \simeq 3.1415926536$ should be entered as `sp.pi`. If you just enter `pi` then that will just be treated as the name of a variable with no special value or properties.

(7.3) The constant $e \simeq 2.718281828$ must be entered as `sp.E` (with a capital E) or `sp.exp(1)`.

(7.4) The square root of minus one can be entered as `sp.I`.

(7.5) In many contexts ∞ can be entered as `sp.oo`.

8. Solving

(8.1) To solve the equation $x^2 + 6 = 5x$ (for example), enter `sp.solve(sp.Eq(x**2+6,5*x))`. If you want to solve for an expression being equal to zero then you do not need `sp.Eq()`, you can just enter `sp.solve(x**2+6-5*x)` for example. Similarly, if you have already defined a function g, you can enter `solve(g(t)=0,t)` to solve the equation $g(t) = 0$. It **does not work** to enter `a == b` instead of `sp.Eq(a, b)`.

(8.2) The above syntax gives the answer in the form $[2, 3]$. If you prefer to have the answer in the form $[\{x : 2\}, \{x : 3\}]$ (which can be more convenient if you want to substitute the result into some other expression), you should instead enter `sp.solve({x^2-5*x+6=0},dict=True)`.

(8.3) The above syntax will give usually complex roots as well as real roots. For example, if we declare `x = sp.symbols('x')` and then enter `sp.solve(x**3+x,x)` we get $[0, -i, i]$. If we only want real roots we can either use `sp.realroots(x**3+x,x)` or we can declare x using `x = sp.symbols('x',real=True)` and then just use `sp.solve()`.

(8.4) To solve several equations in several variables, put the equations in one set of square brackets, and the variables in another set. For example, to solve $2x + 3y = 5$ and $3x + 4y = 7$ for x and y, enter

```
sp.solve({2*x+3*y-5,3*x+4*y-7},{x,y},dict=True);
```

(8.5) If there are no solutions, `sp.solve()` will return the empty list `[]`.

(8.6) If the problem involves trigonometric functions, then there will typically be infinitely many solutions, but `sp.solve()` will only list a few of them. To get all solutions, you can try using `sp.solveset()` instead. For example, we can solve the equation $\cos(x) = 1$ by entering `sp.solveset(sp.cos(x)-1,x)`. The result will be printed as $\{2n\pi \mid n \in \mathbb{Z}\}$, indicating that $\cos(x) = 1$ if and only if x is an integer multiple of 2π. There is quite a long story about how this result is represented internally in Python, which you can read at `https://docs.sympy.org/latest/modules/sets.html`.

(8.7) To get approximate numerical solutions, you can use `sp.nsolve()` instead of `sp.solve()`. For example, you can enter `sp.nsolve(sp.cos(x)-sp.cos(x+1),x,42)` to find a number x with $\cos(x) = \cos(x + 1)$, starting the search at $x = 42$. When using `sp.nsolve()`, you always need to specify the starting point. If you need to solve a large number of equations numerically then

it will usually be best to use the `scipy` or `numpy` packages; you should only use `sp.nsolve()` for occasional numerical solutions when you are mostly working with symbolic equations.

(8.8) Sometimes we need to solve problems like this: find a and b such that $a(x-1)^2+b(x+1)^2 = x$ *for all* x. For this we can enter

```
sp.solve_undetermined_coeffs(a*(x-1)**2+b*(x+1)**2-x,[a,b],x)
```

Similarly, to find u such that $sin(t+u) = cos(t)$ for all t, we could try

```
sp.solve_undetermined_coeffs(sp.Eq(sp.sin(t+u),sp.cos(t)),[u],t)
```

This does not actually work, but we can make it work by giving the solver a hint as follows:

```
sp.solve_undetermined_coeffs(sp.Eq(sp.expand_trig(sp.sin(t+u)),sp.cos(t)),[u],t)
```

9. Standard functions

(9.1) Most functions can be entered using their usual names prefixed with `sp.`, for example `sp.sin(x)` or `sp.ln(y)`. If you do not want to type the prefix, you can change your initial import statement to include

```
from sympy import sin, cos, tan, exp, ln
```

(for example) then you can just enter `sin(x)` instead of `sp.sin(x)`.

(9.2) However, you must always use brackets: enter `sp.tan(x)` instead of `tan x`, and `sp.sin(2*x)` instead of `sin 2x`.

(9.3) The absolute value of x, traditionally written as $|x|$, must be entered as `sp.abs(x)`.

(9.4) Both `sp.ln(x)` and `sp.log(x)` refer to the natural logarithm (to base e) of x.

(9.5) If you want to use the logarithm to base 10 then you should enter `sp.log(x, 10)`. Similarly, enter `sp.log(x, 2)` for $\log_2(x)$ (the logarithm to base 2) and so on.

(9.6) The function e^x should be entered as `sp.exp(x)`.

(9.7) The hyperbolic function $\sinh(x) = (e^x - e^{-x})/2$ can be entered as `sp.sinh(x)`, and similarly for the functions $\cosh(x) = (e^x + e^{-x})/2$ and $\tanh(x) = \sinh(x)/\cosh(x)$.

(9.8) The traditional notation $\sin^2(x)$ refers to the square of $\sin(x)$, which could also be written $(\sin(x))^2$ or $\sin(x)^2$. In Python this must be entered as `sp.sin(x)**2`. Similar comments apply to $\tan^2(x)$ and so on. Note that $\sin(x^2)$ means something different again: it is the sin of the square of x, not the square of the sin.

(9.9) In traditional notation $1/\sin(x)$ is sometimes written as $\csc(x)$ or $\mathrm{cosec}(x)$. In Python `sp.csc(x)` will work, but `sp.cosec(x)` will not. Of course, you can also just enter `1/sp.sin(x)`.

(9.10) In traditional notation $\tan^{-1}(x)$ or $\arctan(x)$ or $\mathrm{atan}(x)$ refers to the angle θ such that $\tan(\theta) = x$. This is completely different from the number $\tan(x)^{-1} = 1/\tan(x)$. In Python you should enter `sp.atan(x)` (**not** `tan^(-1)(x)` or anything like that). Similar comments apply to $\mathrm{asin}(x)$, $\mathrm{atanh}(x)$ and so on.

(9.11) When $x \geq 0$, the notation `sp.sqrt(x)` means, by definition, the positive square root of x. (There is more to say if x is negative or complex, but we pass over that here.) Thus, for example, `sp.sqrt(9)` is definitely 3 and not -3.

10. Defining your own functions

(10.1) If you want to define a simple function like $f(x) = x^2$, enter `f = lambda x: x^2`. It **does not** work to enter `f(x)=x^2;` instead.

(10.2) Similarly, to define the sequence $a(n) = (-1)^n/n$, enter `a = lambda n: (-1)**n/n`, not `a(n)=(-1)**n/n`.

(10.3) It **does not** work to use notation like `f'(x)` for the derivative of $f(x)$; see Section 12 instead.

(10.4) You can define a function of several variables in a similar way. For example, to define $g(a,u,v) = u^a + v^a$, enter `g = lambda a,u,v: u**a+v**a`.

(10.5) For longer and more complicated functions you may prefer to use the more standard Python function definition syntax, like

```
def f(x, y, z):
    return (x+y)**8 + (x-y)**8 + (x+z)**8 + (x-z)**8
```

11. Plotting

In this course we mostly use the `sympy` library, but that does not, by itself, support plotting. We instead use the `numpy` and `plotly` libraries to make plots. (There is also another popular library called

matplotlib which could be used instead of plotly, but on balance we have found plotly to be preferable.) More details of everything discussed here can be found at https://plotly.com/python/.

(11.1) To plot the graph $y = x^3 - x$ from $x = -2$ to $x = 2$ (for example), we would enter

```
x = np.linspace(-2, 2, 201)
y = x**3 - x
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y))
fig.show()
```

The first line sets x to be an array $[x_0, x_1, x_2, \ldots, x_{200}] = [-2, -1.98, -1.96, \ldots, 2]$ consisting of 201 equally spaced points between -2 and 2 (inclusive). The second line sets y to be the array $[y_0, \ldots, y_{200}] = [x_0^3 - x_0, \ldots, x_{200}^3 - x_{200}]$. The third line creates an object called fig representing a blank picture, to which we can add curves, shapes text and so on. In this case we just add a single curve passing through the points with coordinates (x_i, y_i); this is done by the line fig.add_trace(go.Scatter(x=x, y=y)). Finally, the line fig.show() to show the picture that we have constructed. This is not always necessary, if the last line in a cell is related to plotting then Python will often decide for itself that the resulting plot should be shown, but it does not hurt to make it explicit.

(11.2) To label the curve in the above example, we could change the relevant line to

```
fig.add_trace(go.Scatter(x=x, y=y, name='y=x^3-x', showlegend=True))
```

This will display the label as y=x^3-x. If we prefer to have nicely formatted mathematics like $y = x^3 - x$ then we can change the above to say name=r'$y=x^3-x$'. However, this will only work correctly if we have done from header import * as explained in Section 1.

(11.3) Various features of the plotted curve can be changed by adding extra arguments inside go.Scatter(). For example, if we enter

```
fig.add_trace(go.Scatter(x=x, y=y, line_color='red', line_dash='dot', line_width=5))
```

then the curve will be shown as a thick red dotted line. Colours can be entered as names like orange or as hex codes (like #40e0d0 for turquoise; an internet search will find other examples). Note that American spelling (color not colour) is required.

(11.4) Various features of the overall picture can be changed using fig.update_layout() (before fig.show()). For example, we can insert

```
fig.update_layout(height=400, width=800, xaxis_range=[-3,3], yaxis_range=[0,10])
```

to ensure that the plot is 800×400 pixels and shows x values from -3 to 3 and y values from 0 to 10.

(11.5) To make the vertical scale the same as the horizontal scale, add the options yaxis_scaleanchor="x",yaxis_scaleratio=1 in fig.update_layout(). Alternatively, call equal_aspect(fig) before fig.show(). (This is not a standard part of plotly and relies on having called from header import * at the top of the notebook.)

(11.6) To plot more than one graph in the same picture, we can just call fig.add_trace() several times. For example, we can plot $y = x^3 - x$ and $y = x^3 + x$ in the same picture as follows:

```
x = np.linspace(-2, 2, 201)
y0 = x**3 - x
y1 = x**3 + x
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y0))
fig.add_trace(go.Scatter(x=x, y=y1))
fig.show()
```

If we do not specify the colours, they will be chosen automatically so that the two curves have different colours. Similarly, to plot $\cos(t)$ and $\sin(t)$ together from $t = -3\pi$ to $t = 3\pi$, enter

```
t = np.linspace(-3*np.pi, 3*np.pi, 201)
fig = go.Figure()
fig.add_trace(go.Scatter(x=t, y=np.cos(t)))
fig.add_trace(go.Scatter(x=t, y=np.sin(t)))
fig.show()
```

(11.7) In the above example, there are tick marks on the x-axis at $x = -8$, $x = -7$, $\ldots, x = 8$. It would be more natural to have the tick marks at $x = -3\pi$, $x = -2\pi$, $\ldots, x = 3\pi$. We can achieve this by calling use_pi_ticks(fig,-3,3) before fig.show(). (This is not a standard

part of plotly and relies on having called `from header import *` at the top of the notebook.) For ticks with a spacing of $\pi/2$ instead of π, we can call `us_pi_ticks(fig,-3,3,2)`.

(11.8) To make the x and y axes invisible (together with the associated tick marks and labels) add the options `xaxis_visible=False, yaxis_visible=False` in `fig.update_layout()`.

(11.9) If we plot a function with discontinuities, like $y = 1/x$, then by default we will get spurious vertical lines at the discontinuity points. This can be avoided as follows:

```
x = np.linspace(-2,2,400)
y = 1/x
jumps = np.where(np.abs(np.diff(y)) >= 0.5)[0]
x = np.insert(x, jumps+1, np.nan)
y = np.insert(y, jumps+1, np.nan)
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y, mode='lines',
                         name=r'$y = \{x\}$', showlegend=True))
fig.update_layout(height=400, width=800, yaxis_range=[-8,8], showlegend=True)
```

The third line finds the values of x where y jumps by more than 0.5, indicating a discontinuity. (The value 0.5 might need to be adjusted to give the right outcome in other examples.) The next two lines insert the value `np.nan` (indicating "not a number") at the corresponding places in the arrays x and y. The plotting system takes this as an indication that the curve should be broken without spurious vertical lines.

(11.10) To improve the accuracy of a graph, just increase the third argument in `np.linspace`. For example, the code

```
x = np.linspace(-1, 1, 50)
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=x * np.sin(np.pi / x)))
```

gives a very jagged picture, but it becomes much better if we change `np.linspace(-1,1,50)` to `np.linspace(-1,1,1000)`.

(11.11) To plot the curve given parametrically by $x = 1/(1+t^2)$ and $y = \sin(t)$ from $t = -5$ to $t = 5$ (for example), enter

```
t = np.linspace(-5, 5, 400)
x = 1/(1 + t ** 2)
y = np.sin(t)
fig = go.Figure()
fig.add_trace(go.Scatter(x=x, y=y))
```

Similarly, to plot the curve $(x, y) = (\sin(2t), \cos(3t))$ for $t = 0$ to 2π, enter

```
t = np.linspace(0, 2*np.pi, 400)
fig = go.Figure()
fig.add_trace(go.Scatter(x=np.sin(2*t), y=np.cos(3*t)))
```

(11.12) Consider a curve given implicitly, say by the equation $x + y + x^2y^2 = 1$ with $-3 \leq x \leq 3$ and $-4 \leq y \leq 4$. This can be plotted as follows:

```
x0 = np.linspace(-3,3,100)
y0 = np.linspace(-4,4,100)
x, y = np.meshgrid(x0, y0)
z = x + y + x**2 * y**2
fig = go.Figure()
fig.add_trace(go.Contour(x=x0, y=y0, z=z,
                         contours=dict(type='constraint', value=1),
                         line_color='red'))
fig.update_layout(height=600, width=600, xaxis_range=[-3,3], yaxis_range=[-4,4])
```

The first line sets `x0` to ba a one-dimensional array of x-values, and similarly for the second line. The third line sets x to be a two-dimensional array of x-values, and y to be a two-dimensional array of y-values. The fourth line uses these to compute a two-dimensional array z of values of the function $x + y + x^2y^2$, then the sixth line draws the curve where $z = 1$. Similarly, you can plot the circle $x^2 + y^2 = 4$ like this:

```
x0 = np.linspace(-2,2,100)
y0 = np.linspace(-2,2,100)
x, y = np.meshgrid(x0, y0)
z = x**2 + y**2 - 1
fig = go.Figure()
fig.add_trace(go.Contour(x=x0, y=y0, z=z,
                         contours=dict(type='constraint', value=0),
                         line_color='red'))
fig.update_layout(height=600, width=600, xaxis_range=[-2,2], yaxis_range=[-2,2])
```

(11.13) We might want to produce a basic picture, and then later generate some further plots which combine the basic one with some extra elements. For this we can make the basic picture by calling `basic_fig=go.Figure()` and then `basic_fig.add_trace(...)`. Then we can call `new_fig=go.Figure(basic_fig)` to make a new figure which is a copy of the basic one, and `new_fig.add_trace(...)` to add extra elements to the new figure, and `new_fig.show()` to display the result.

(11.14) Alternatively, if we have already created figures `fig0` and `fig1`, we can make a new figure that combines them by entering `fig = go.Figure(data = fig0.data  fig1.data)+`.

(11.15) By default, `go.Scatter(x=u, y=v)` will produce a connected curve passing through the points (u_i, v_i). Sometimes we might instead just want to show the points (u_i, v_i) without connecting points. For this, we need to add the option `mode='markers'` to `go.Scatter()`. For example, we can display the points $(11, 50), (22, 40), (33, 30), (44, 20), (55, 10)$ as follows:

```
fig = go.Figure()
fig.add_trace(go.Scatter(x=[11,22,33,44,55],y=[50,40,30,20,10], mode='markers'))
```

Often we just want to display the y-values, and the x-values are not important, we just take them to be $0, 1, \ldots, n-1$. In that context, we do not actually need to specify the x-values, we can just call something like `fig.add_trace(go.Scatter(y=[50,40,30,20,10], mode='markers'))` and the x-values will be inserted automatically.

12. Differentiation

(12.1) To differentiate $x + \sin(x)$ with respect to x (for example), enter `diff(x+sin(x),x)`. This is equivalent to $\frac{d}{dx}(x + \sin(x))$ in traditional notation. Similarly, to differentiate $1/(1 - t^2)$ with respect to t we enter `diff(1/(1-t^2),t)`.

(12.2) To differentiate y three times with respect to x (for example), enter `diff(y,x,3)`. This is equivalent to d^3y/dx^3 in traditional notation. Similarly, to find the second derivative of $\sin(\theta)$ with respect to θ we enter `diff(sin(theta),theta,2)`.

(12.3) If x and y are related implicitly by an equation (say $x + y + x^2y^2 = 1$) then you can find dy/dx in terms of x and y by implicit differentiation, like this:

```
sp.idiff(x+y+x^2*y^2-1,y,x)
```

(12.4) Sometimes we just want to mention the derivative without actually working it out. We can do this by adding the option `evaluate=False`. For example, entering
`u = sp.diff(x**2, x, evaluate=False)` gives an object u which is displayed as $\frac{d}{dx}x^2$. If you then enter `u.doit()` then sympy will evaluate the derivative and return $2x$. Similarly, `v = sp.diff(t**10,t,2)` prints as $\frac{d^2}{dt^2}t^{10}$, whereas `v.doit()` or `sp.diff(t**10,t,2)` gives $90t^8$.

(12.5) To find the Taylor series of a function $f(x)$ near a point $x = a$ to order n, enter `sp.series(f(x),x,a,n)`. Note that "order n" means that the highest term allowed is a multiple of x^{n-1}, and all terms x^n and higher are discarded. For example, `sp.series(1/x,x,2,7)` gives the Taylor series for $1/x$ near $x = 2$ including terms up to $(x - 2)^6$.

(12.6) The command `p = sp.series(...)` command gives an answer including a term like $O(x^7)$, to indicate that there are infinitely many more terms, starting with a multiple of x^7. We cannot do very much with this answer until we have converted it to an ordinary polynomial (without the $O(x^7)$ term on the end). We can do this by entering `p = p.removeO()`.

13. Integration

(13.1) To calculate an indefinite integral like $\int \tan(x)\,dx$, enter `sp.integrate(sp.tan(x),x)`.

(13.2) Sympy's answer will never include an arbitrary constant '$+c$'. In some contexts (particularly, solution of differential equations) the constant makes a difference and must be included. In other contexts it is irrelevant; sympy cannot help you to decide about this.

(13.3) To calculate a definite integral like $\int_1^5 t^3\,dt$, enter `sp.integrate(t**3,(t,1,5))`. Similarly, to calculate $\int_0^2 \sqrt{1+x}\,dx$, enter `sp.integrate(sp.sqrt(1+x),x=0..2)`.

(13.4) It is also possible to have infinite limits. For example, $\int_{-\infty}^{\infty}(1+x^4)^{-1}dx$ can be entered as

```
sp.integrate(1/(1+x**4),(x,-sp.oo,sp.oo))
```

(13.5) As with differentiation, it is sometimes useful to mention an integral without evaluating it. You can do this by writing `sp.Integral()` instead of `sp.integrate()`. For example, if you enter `u = sp.Integral(x^2,x)` then you will get an object that is displayed as $\int x^2\,dx$; if you then enter `u.doit()`, you will get $x^3/3$.

14. Sequences and sums

(14.1) To generate a finite sequence of terms, we can use standard Python list comprehension. For example, the sequence $1/2, 1/4, 1/6, 1/8, 1/10$ is the sequence of terms $1/(2k)$ for $k = 1, 2, 3, 4, 5$; it can be generated by the command

```
list(sp.Rational(1,2*k) for k in range(1,6))
```

Remember here that `range(1,6)` refers to the range from 1 to 6 including 1 but not including 6, so it gives $1, 2, 3, 4, 5$ as required.

(14.2) For a finite list u as above, we can just enter `sum(u)` to get the sum of all the entries in the list. For example, `u=list(10**k for k in range(5))` gives $u = [1, 10, 100, 1000, 10000]$ and then `sum(u)` gives 11111. Moreover, `sum(10**k for k in range(5))` will also work for this; it is not necessary to make a list as an intermediate step.

(14.3) However, we might want to get a general formula for a sum like $\sum_{k=0}^{n} 3^{-k}$ in terms of n, and this cannot be done by the above method. Instead, we need to declare n and k to be integer symbols and use `sp.Sum()`, like this:

```
k, n = sp.symbols('k n', integer=True)
u0 = sp.Sum(3**(-k),(k,0,n)).doit()
```

Note that the naming and behaviour is a bit inconsistent with the naming and behaviour for integrals. The function `sp.Sum()` always returns an unevaluated sum which is displayed as something like $\sum_{k=0}^{n} 3^{-k}$; it is always necessary to use `.doit()` to find the actual value of the sum. Also, the range `(k,0,n)` means that k runs from 0 to n inclusive, which is consistent with tradional mathematical notation, but not with the usual Python convention for ranges.

(14.4) We can also handle some infinite sums. For example, it is a well-known result that $\sum_{k=1}^{\infty} k^{-2} = \pi^2/6$, and we can do this in sympy by entering `sp.sum(k**(-2),(k,1,sp.oo))`.

15. Some common problems

(15.1) All abstract symbols need to be introduced using somthing like `x = sp.symbols('x')`. If you try to use x without having declared it, you will get an error like `name 'x' is not defined`. It can also happen that you give x a value, perhaps by entering $x = 42$, and later you forget that you did that, and try to treat x as an abstract symbol. This might give you any of a range of different error messages, or it might just give you an answer that is incorrect or meaningless but with no error message. You can check the status of x by entering `type(x)`. If x has been declared as an abstract symbol, then `type(x)` should give `sympy.core.symbol.Symbol`. If x has been set equal to 42, then `type(x)` should give `int`. If x has not been introduced in any way, then `type(x)` will give an error. If x has been given a value that is no longer relevant, you can enter `del x` to remove it.

(15.2) Remember `*`'s for multiplication.

- Enter $2(x+y)$ as `2*(x`y`)`+. If you instead enter `2(x+y)` then you may get a cryptic error message saying `'int' object is not callable`.
- Enter xy as $x*y$. If you instead enter `xy` then you will usually get an error message `name 'xy' is not defined`.
- For the product of a and $x+y$, enter `a*(x+y)`. To evaluate a function $f(t)$ at the point $t = x+y$, enter $f(x+y)$. If a is an object for which `a*(x`y`)`+ makes sense, then $a(x+y)$ will not make sense and will give an error. If f is an object for which $f(x+y)$ makes sense, then $f*(x+y)$ will not make sense and will give an error.

(15.3) $\frac{a+b}{c+d}$ must be entered as `(a+b)/(c+d)` (not `a+b/c+d`), and $\frac{a}{(u+v)(x+y)}$ must be entered as `a/((u+v)*(x+y))` (not `a/(u+v)*(x+y)`).

(15.4) $a^{1/4}$ must be entered as `a^sp.Rational(1,4)`, not `a^(1/4)` or `a^1/4`.

(15.5) Always use round brackets (not square or curly ones) for algebraic grouping. Square brackets are used for lists or for subscripts (e.g. x_5 is `x[5]`). Curly brackets are used for sets.

(15.6) Remember to remove definitions when you have finished with them, by using `del` or clicking the `Restart;` button. If you enter an expression like $(x+y)^5$ and Maple converts it to something completely unrelated like $(\sin(\theta)+e^{-u})^5$, the most likely reason is that you previously defined `x=sp.sin(theta);` and `y=sp.exp(-u);`, and you forgot to remove these definitions. You can enter `del x,y` to do that.

(15.7) Remember the constant $e = 2.71828\ldots$ must be entered as `sp.E` or `sp.exp(1)`, and e^x must be entered as `exp(x)`. If you enter `e ** x` instead of $sp.exp(x)$, you will most often get an error message `name 'e' is not defined`. However, if you happen to have defined e, you might get some other behaviour.

(15.8) Remember that the constant $\pi = 3.14159\ldots$ must be entered as `sp.pi`, not `pi`. If sympy is doing funny things like refusing to simplify $\sin(\pi)$ to 0, it may be that you entered `pi` instead of `sp.pi` by mistake. You can cure this by entering `pi=sp.pi`.